

ADVENTIST UNIVERSITY OF CENTRAL AFRICA

Faculty of Information Technology

Department of Software Engineering

E-Water Management System (EWMS)

Final Exam Project Documentation

Field	Details
Student Name	Alvin MUNYAMBO
Student Id	27146
Instructor	RUTARINDWA JEAN PIERRE
Academic Year	2025 - 2026
Topic	E-Water Management System (EWMS)

Phase 1 — System Analysis and Design

i. General Description and Analysis of the Case Study

WASAC (Water and Sanitation Corporation) is the national public utility of Rwanda responsible for the production, treatment, and distribution of clean drinking water to millions of Rwandan households and businesses. Headquartered in Kigali, WASAC manages thousands of water connection points and employs field agents, billing staff, and customer service personnel across the country.

As WASAC's customer base grew, management realized that their manual, paper-based operations were no longer efficient. Field agents physically visited properties each month to record meter readings on paper. Bills were calculated by hand. Payments through MTN MoMo and Airtel Money were reconciled manually. This led to the development of the E-Water Management System (EWMS) — a fully digital web platform to automate all WASAC operations.

Module	Description
Client Management	Register homeowners, assign meter IDs, manage account status (active/suspended)
Meter Readings	Admin records monthly water usage in litres for each client meter
Billing	System generates monthly bills with amount in RWF and due date
Payments	Clients pay bills online via MTN MoMo or Airtel Money simulation
Dashboards	Admin sees real-time stats; clients see usage charts and current balance
Access Control	JWT-based role authentication separating Admin portal and Client portal

ii. Functional Diagram

The diagram below shows how WASAC's internal departments work and how EWMS connects all their operations into one centralized digital platform.

- * WASAC field agents enter monthly meter readings through the Admin Portal instead of paper forms
- * The billing module automatically generates monthly invoices for each registered client
- * Homeowners log in to the Client Portal to view their water usage history and pay bills online
- * Finance staff track all payment records in real time without manual mobile money reconciliation
- * The admin dashboard provides live statistics on active accounts, unpaid bills, meter status, and revenue
- * All data is stored in a MongoDB database running inside a Docker container

Department	Old Manual Process	New EWMS Digital Solution
Field Department	Paper meter readings — data loss and errors	Digital meter entry via Admin Portal

Department	Old Manual Process	New EWMS Digital Solution
Billing Department	Manual invoice calculation — 6 to 8 week cycles	Instant automated bill generation
Finance Department	Mobile money reconciliation done by hand	Automated payment recording and tracking
Customer Service	Clients visit office for every query	Self-service Client Portal with full history

iii. Problem Statement

Before EWMS was developed, WASAC relied entirely on manual paper-based processes. This caused six major problems:

1. Manual Meter Readings

Field agents recorded water usage on paper forms every month. Transcription errors, lost forms, and illegible handwriting led to incorrect bills and customer disputes that were difficult to resolve.

2. Paper-Based Billing Cycles

Bills were calculated manually and delivered by hand, resulting in billing cycles of 6 to 8 weeks. Customers frequently received incorrect bills with no easy digital way to check or dispute them.

3. No Real-Time Payment Tracking

Payments via MTN MoMo and Airtel Money were not automatically matched to customer accounts. Finance staff had to manually reconcile mobile money records, causing delays and accounting errors.

4. No Client Self-Service

Customers had no way to check their water usage, outstanding balance, or payment history without physically visiting a WASAC office, resulting in long queues and poor customer experience.

5. Manual Account Disconnections

Suspending and reconnecting accounts for non-payment required internal paper memos and field visits, causing significant delays in enforcement and inconsistent disconnection policies.

6. No Management Analytics

WASAC management had no real-time visibility into total active accounts, unpaid bills, revenue trends, or disconnected meters, making financial planning and reporting very difficult.

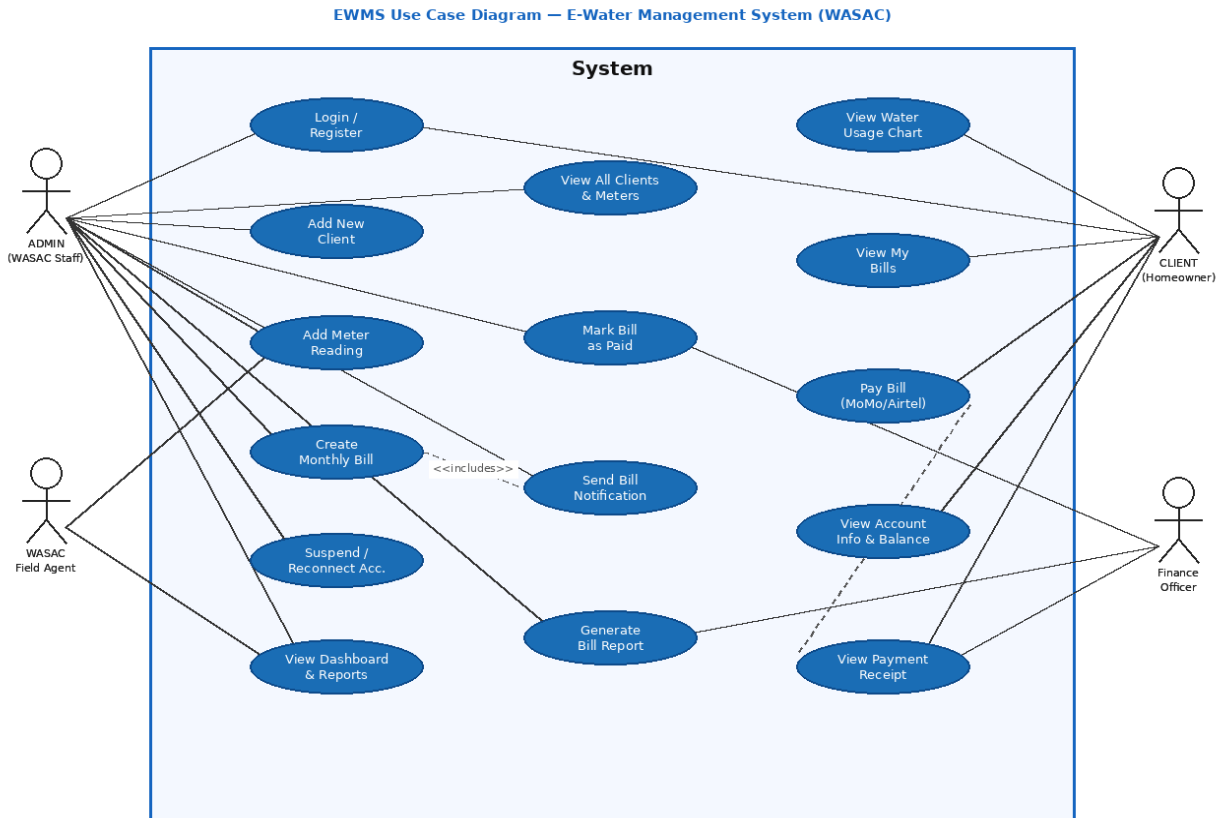
Proposed Solution: The EWMS was developed to solve all six problems by providing a centralized, role-based digital platform with automated billing, online payments, real-time dashboards, and a self-service client portal.

iv. Object-Oriented System Analysis and Design

The following UML diagrams were produced using Object-Oriented System Analysis and Design methodology.

1. Use Case Diagram

Actors: Admin (WASAC Staff), WASAC Field Agent, Client (Homeowner), Finance Officer



Actor	Use Cases
Admin (WASAC Staff)	Login, Add New Client, Add Meter Reading, Create Monthly Bill, View All Clients/Bills/Meters, Mark Bill Paid, Suspend/Reconnect Account, View Dashboard Stats
WASAC Field Agent	Login, Add Meter Reading, View Dashboard Stats
Client (Homeowner)	Register, Login, View Water Usage Chart, View My Bills, Pay Bill (MoMo/Airtel), View Account Info, View Payment Receipt
Finance Officer	Login, Mark Bill as Paid, Generate Bill Report, View Payment Receipt

The <<include>> relationship: Create Monthly Bill always triggers Send Bill Notification. The <<extend>> relationship: Pay Bill optionally triggers Send Payment Receipt.

2. Class Diagram

The Class Diagram defines five core classes and their relationships:

Class	Key Attributes	Key Methods
User	_id, name, email, phone, passwordHash, meterId, isActive, isVerified	comparePassword(), register(), login() : JWT
Admin	_id, name, email, passwordHash, role	comparePassword(), login() : JWT
Meter	_id, meterId, userId, isActive, balance (RWF), monthlyUsage[]	getUsageHistory(), updateReading()
Bill	_id, userId, meterId, amount (RWF), month, year, dueDate, status	generateBill(), markPaid()
Payment	_id, userId, billId, amount, method (momo airtel), phone, status, reference	processPayment(), getReference()

Relationships:

- * User (1) to Meter (1) — each homeowner owns exactly one water meter
- * Meter (1) to Bill (many) — one meter generates one bill every month
- * Bill (1) to Payment (1) — each paid bill has exactly one payment record linked to it
- * Admin manages all Users, Meters, Bills, and Payments through the Admin Portal

3. Activity Diagram

The Activity Diagram below models the complete bill payment flow from login to payment confirmation:

- * Client opens the EWMS portal and enters their email and password
- * System validates the credentials — if wrong: show error message and stay on the login page
- * If credentials are correct: load the dashboard showing usage chart, balance, and pending bill
- * System checks whether the client has an unpaid bill — if no bill: show "No pending bills" message
- * If there is an unpaid bill: display the bill amount, due date, and meter ID to the client
- * Client selects their payment method: MTN MoMo or Airtel Money
- * Client enters their phone number and clicks the Pay Now button
- * The API creates a Payment record in the database with status "confirmed"
- * The Bill status is automatically updated from "pending" to "paid" in the database
- * The system displays a unique payment reference number (EWMS-timestamp) to the client as confirmation

4. Sequence Diagram

The Sequence Diagram shows the exact order of messages exchanged between four system components when the Admin creates a monthly bill:

Step	From	To	Message
1	Admin Browser	Frontend JS	Admin opens the Bills page and clicks Create Bill
2	Frontend JS	Node.js API	GET /api/admin/bills with JWT token in Authorization header
3	Node.js API	Node.js API	Verify JWT token and confirm that role equals admin
4	Node.js API	MongoDB	Bill.find() — retrieve all existing bill documents
5	MongoDB	Node.js API	Return array of bill documents
6	Node.js API	Frontend JS	200 OK response with bills array
7	Frontend JS	Admin Browser	Render the bills table in the browser
8	Admin Browser	Frontend JS	Admin fills the Create Bill form and clicks Submit
9	Frontend JS	Node.js API	POST /api/admin/create-bill with userId, meterId, month, year, amount, dueDate
10	Node.js API	Node.js API	Verify JWT again and confirm admin role before proceeding
11	Node.js API	MongoDB	Bill.create() — save the new bill document to the database
12	MongoDB	Node.js API	Return confirmation that the bill was saved successfully
13	Node.js API	Frontend JS	201 Created response with the new bill object
14	Frontend JS	Admin Browser	Show success notification and automatically refresh the bills table

5. Component Diagram

The system is structured as a three-tier containerized architecture running inside Docker:

Container	Technology	Port	Role
ewms_frontend	Nginx serving HTML/CSS/JS	localhost:8090	Delivers all web pages to the browser (login, dashboard, admin portal)
ewms_backend	Node.js + Express REST API	localhost:5000	Handles all business logic, JWT authentication, and API requests
ewms_mongo	MongoDB 7 database	Internal only	Stores all data: users, admins, meters, bills, payments

All three containers run on an isolated Docker bridge network called ewms_net. They communicate with each other using service names instead of IP addresses. The frontend talks to the backend via HTTP and JSON. The backend talks to MongoDB via Mongoose ORM. No container is accessible from outside the network except through the mapped ports (8090 and 5000).

Phase 2 — Software Prototype and Design Pattern

Software Development Prototype

The EWMS prototype is a fully functional three-tier web application. It covers all core WASAC operations including client registration and management, meter reading recording, automated billing, online bill payment, and real-time reporting dashboards for both administrators and homeowners.

Layer	Technology	Role
Frontend	HTML5, CSS3, Vanilla JavaScript	Client and Admin portals served by Nginx web server
Backend	Node.js + Express.js	REST API handling all business logic and JWT authentication
Database	MongoDB 7 + Mongoose ORM	Persistent storage for 5 collections: users, admins, meters, bills, payments
Authentication	JSON Web Tokens (JWT) + bcrypt	Secure role-based access separating Admin and Client access
Containerization	Docker + Docker Compose	Three isolated containers deployed on a shared internal network

Design Pattern Used: Repository Pattern

What is the Repository Pattern?

The Repository Pattern is a structural design pattern that creates a clear separation between the data access layer (the MongoDB database queries) and the business logic layer (the route handlers that process requests). Instead of writing database queries directly inside the route files, all database operations are moved into dedicated model files that act as repositories. The routes simply call methods on the models without knowing anything about how the database works.

Why was the Repository Pattern chosen?

- * It separates database logic from API route handlers, making each file responsible for only one thing
- * It makes the code much easier to read, maintain, and test
- * If the database ever needs to be changed, only the model files need to be updated
- * It follows the Single Responsibility Principle from Google Node.js best practices
- * It hides sensitive operations like password hashing inside the model where they cannot be bypassed

How the Repository Pattern is applied in EWMS

Repository (Model File)	Database Collection	What it handles
User.js	users	Homeowner registration, login, password comparison using bcrypt, account status

Repository (Model File)	Database Collection	What it handles
User.js (Admin schema)	admins	Admin login, password comparison, role verification for JWT
Meter.js (Meter schema)	meters	Recording meter readings, updating balance, retrieving usage history
Meter.js (Bill schema)	bills	Generating monthly bills, marking bills as paid, retrieving bills by user
Meter.js (Payment schema)	payments	Recording payment transactions, generating unique reference codes

Practical example: When a client logs in, the route handler in `routes/auth.js` calls `user.comparePassword(password)`. The route has absolutely no knowledge of `bcrypt` or how password hashing works. The User model (repository) handles all that complexity internally. This prevents security vulnerabilities from being accidentally introduced in the route files.

Programming Best Practices Applied

Best Practice	How it is applied in EWMS
Meaningful names	camelCase used throughout: <code>userId</code> , <code>meterId</code> , <code>passwordHash</code> , <code>monthlyUsage</code> , <code>isActive</code>
Proper indentation	2-space indentation consistently applied in all JavaScript files
Modular files	Each feature has its own route file: <code>auth.js</code> , <code>bills.js</code> , <code>meters.js</code> , <code>payments.js</code> , <code>admin.js</code>
Single responsibility	Models handle data shape, routes handle logic, middleware handles authentication
Error handling	Every async route handler uses <code>try/catch</code> with meaningful HTTP status codes and messages
Environment variables	<code>.env.example</code> documents all required secrets; the real <code>.env</code> is excluded via <code>.gitignore</code>
Security	Passwords hashed with <code>bcrypt</code> (<code>salt rounds=10</code>); admin routes protected by <code>adminMiddleware</code>
REST conventions	GET for reading data, POST for creating, PATCH for updating — correct HTTP methods used
Code comments	Important functions and business logic are explained with inline comments
Least privilege	Client JWT tokens cannot access admin routes — the role is verified on every request

Prototype Screenshots

The screenshots below show the key screens of the running EWMS application. Take screenshots from `http://localhost:8090` and insert them in the boxes.

localhost:8090/pages/admin-login.html



Admin Portal

WASAC Staff Access - E-Water Management System

Staff email

admin@wasac.rw

Password

Staff password

Admin sign in

[← Back to homeowner login](#)

[SCREENSHOT] Admin Dashboard — Stats cards: Total Clients, Active Meters, Unpaid Bills, Disconnected

The screenshot shows the EWMS Admin Dashboard. The browser address bar indicates the URL is localhost:8090/pages/admin-dashboard.html. The dashboard header includes the EWMS Admin · WASAC logo and the user Alvin Munyambo with a Sign out link. The left sidebar contains a MANAGEMENT section with Overview, Clients, Bills, and Meters, and a SYSTEM section with Sign out. The main content area is titled 'Admin overview' and 'WASAC E-Water Management System'. It features four stats cards: TOTAL CLIENTS (6 Registered homeowners), ACTIVE METERS (5 Online devices), UNPAID BILLS (6 Pending payment), and DISCONNECTED (1 Suspended accounts). Below these is a 'Quick actions' section with a large blue button '+ Add new client' and two smaller buttons: 'Create bill' and 'Add meter reading'.

EWMS Admin · WASAC

Alvin Munyambo Sign out

MANAGEMENT

- Overview
- Clients
- Bills
- Meters

SYSTEM

- Sign out

Admin overview

WASAC E-Water Management System

TOTAL CLIENTS
6
Registered homeowners

ACTIVE METERS
5
Online devices

UNPAID BILLS
6
Pending payment

DISCONNECTED
1
Suspended accounts

Quick actions

+ Add new client

Create bill

Add meter reading

[SCREENSHOT] Admin — Clients List showing Alvin MUNYAMBO with Active badge

The screenshot shows the EWMS Admin Dashboard with the 'Clients' section selected in the sidebar. The header includes the EWMS Admin · WASAC logo and the user Alvin Munyambo with a Sign out link. The main content area is titled 'Manage homeowner accounts'. It features a table of all clients with columns for NAME, EMAIL, PHONE, METER ID, STATUS, and ACTIONS. The table lists six clients, with Alvin MUNYAMBO highlighted in blue. The STATUS column shows 'Active' for the first five clients and 'Suspended' for Alvin MUNYAMBO. The ACTIONS column contains 'Disconnect' buttons for the first five clients and a 'Reconnect' button for Alvin MUNYAMBO.

EWMS Admin · WASAC

Alvin Munyambo Sign out

Overview

Manage homeowner accounts

All clients

Search...

NAME	EMAIL	PHONE	METER ID	STATUS	ACTIONS
Jean Baptiste SIBONIMANA	jeanbaptiste1@gmail.com	0788121278	WASAC - 2026 - 00005	Active	Disconnect
Beline MUREKATETE	belinem5@gmail.com	0788546789	WASAC - 2026 - 00006	Active	Disconnect
Alice MUSONI	musonialice@gmail.com	0788232343	WASAC - 2026 - 00003	Active	Disconnect
Chris GANZA	chrisg2@gmail.com	0788545434	WASAC - 2025 - 00002	Active	Disconnect
Patrick MUGABO	patmugabo12@gmail.com	0788656545	WASAC - 2026 - 00001	Active	Disconnect
Alvin MUNYAMBO	munyambo04@gmail.com	+250781384024	WASAC - 2025 - 33445	Suspended	Reconnect

[SCREENSHOT] Admin — Add New Client modal form

EWMS Admin · WASAC

MANAGEMENT

Overview

Clients

Bills

Meters

ITEM

Sign out

Clients

Manage homeowner accounts

All clients

NAME
Bruce Manzi
Richard Mugabo
Bizimana Bosco
xxx
Alvin MUNYAMBO

+ Add new client

Full name

Your Name

Email

Email Address

Phone

Phone Number

Meter / Card ID

WASAC-2024-XXXXX

Password

Password

Initial balance (RWF)

0

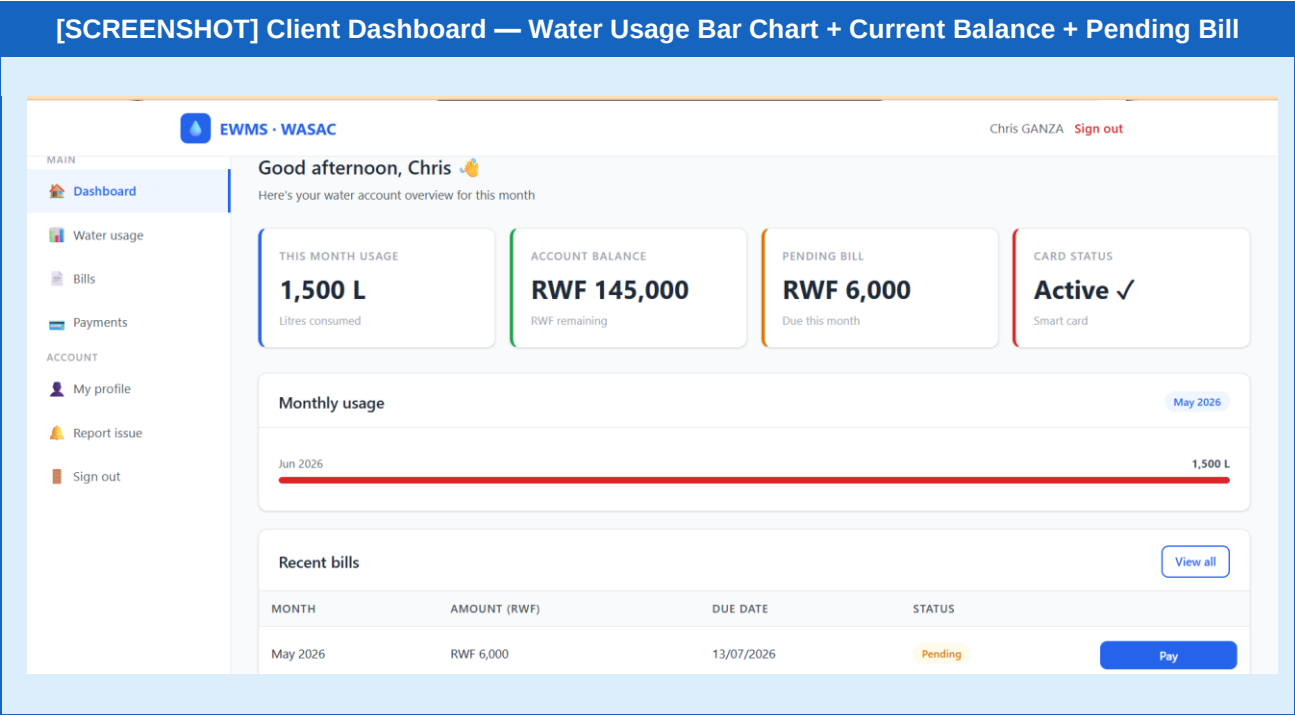
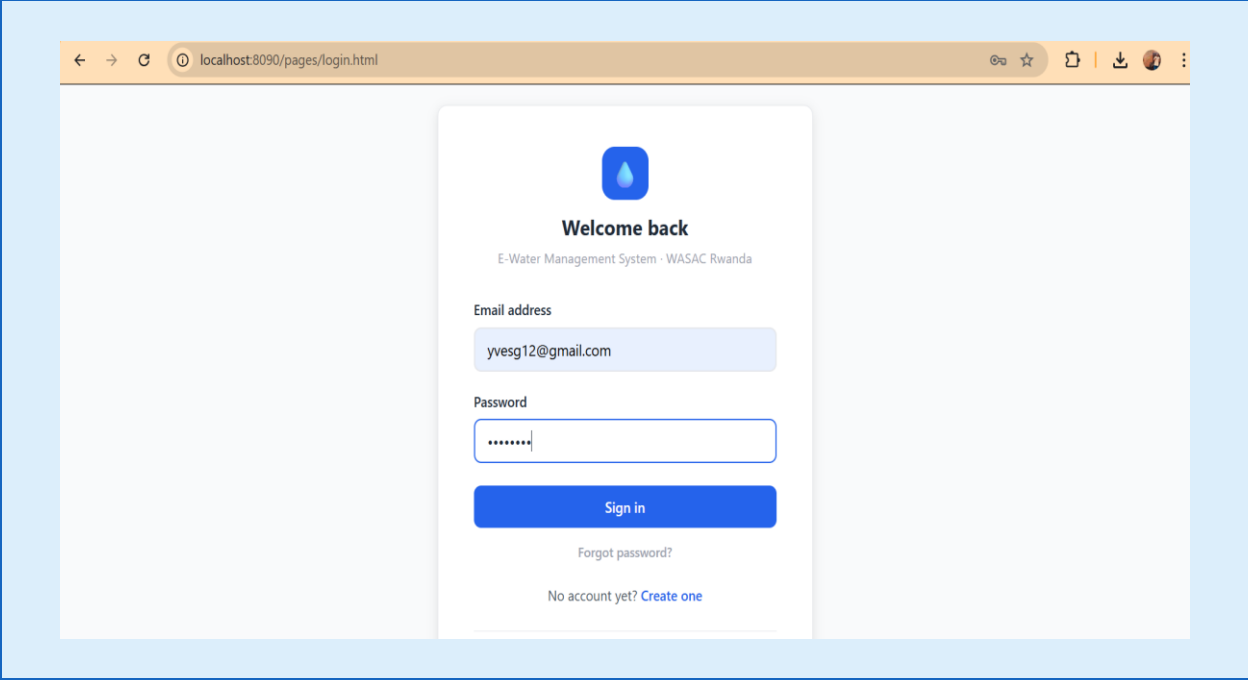
Alvin Munyambo Sign out

ent

Search...

ID	STATUS	ACTIONS
-2026-0001	Active	Disconnect
-2026-11110	Active	Disconnect
-2026-11113	Active	Disconnect
-2026-0000	Suspended	Reconnect
-2025-33445	Active	Disconnect

[SCREENSHOT] Client ogin Page — <http://localhost:8090/pages/login.html>



[SCREENSHOT] Client — Pay Bill screen showing MTN MoMo and Airtel Money options

The screenshot displays the 'Pay Bill' screen of the EWMS - WASAC application. A modal window is open for payment. The modal contains the following fields and options:

- Pay bill** (Title)
- Choose your payment method** (Instruction)
- Amount (RWF)**: RWF 6,000
- Payment method**: MTN Mobile Money (selected from a dropdown)
- Phone number**: 07845455343
- Buttons**: Cancel, Pay now

The background interface shows the user's profile (GANZA Yves, Sign out), water usage (1,500 L), and bill details (May 2026, 1,500 L). The user's card status is 'Active'.

Phase 3 — Docker and Version Control

Part 1: What is Docker and Why Does it Matter?

Imagine you build an application on your laptop and it works perfectly. But when you send it to someone else or try to run it on a different computer, it breaks because that computer has a different version of Node.js, or is missing a library, or has different settings. This is the classic "it works on my machine" problem that developers have struggled with for years.

Docker solves this problem completely. Docker packages your entire application — the code, the Node.js runtime, all the libraries, the configuration files, and even the operating system layer — into a single unit called a container. This container runs exactly the same way on any computer, any server, or any cloud platform. It does not matter what operating system the host machine has.

Think of a Docker container like a shipping container at a port. The shipping container holds everything needed — the goods, the packaging, the label — and it can be loaded onto any ship, truck, or train in the world and it will arrive in exactly the same condition. Docker works the same way for software.

Docker Concept	Simple Explanation	EWMS Example
Image	A blueprint or recipe for creating a container. Read-only.	The Dockerfile in the backend folder builds the Node.js image
Container	A running instance of an image. This is the actual live application.	ewms_backend, ewms_frontend, ewms_mongo are all running containers
Dockerfile	A text file with step-by-step instructions to build an image	backend/Dockerfile tells Docker how to set up the Node.js server
docker-compose.yml	A file that defines and starts multiple containers together at once	One command (docker-compose up) starts all 3 EWMS containers
Volume	Storage that stays alive even when a container is stopped or restarted	mongo_data volume keeps your database data safe between restarts
Network	An isolated virtual network that lets containers talk to each other	ewms_net connects frontend, backend, and MongoDB privately

Part 2: How EWMS was Dockerized — Step by Step

Step 1: The Backend Dockerfile

A Dockerfile is a script of instructions that tells Docker exactly how to build an image for your application. The EWMS backend uses a two-stage build to keep the final image as small and secure as possible.

Stage 1 (the builder stage): Docker downloads a small Alpine Linux image with Node.js 20 installed. It then copies the package.json file and runs npm install to download all the required libraries. This stage does the heavy work of installing dependencies.

Stage 2 (the final stage): Docker creates a fresh, clean Alpine Linux image with Node.js. It copies only the installed node_modules from Stage 1 (not the dev tools). It then copies all the application code. The application runs as a non-root user called "node" for security — this means even if someone somehow broke into the container, they would not have administrator permissions. A health check is configured so Docker automatically restarts the container if the server stops responding.

Dockerfile Instruction	What it Does in Simple Terms
FROM node:20-alpine AS builder	Download a small Linux system with Node.js 20 already installed
WORKDIR /app	Set the working folder inside the container to /app
COPY package*.json ./	Copy the package files first so Docker can cache the npm install step
RUN npm install --omit=dev	Install only the production libraries, skip development tools
FROM node:20-alpine	Start a fresh clean image for the final stage (no build tools)

Dockerfile Instruction	What it Does in Simple Terms
<code>COPY --from=builder /app/node_modules</code>	Copy the installed libraries from the builder stage
<code>COPY . .</code>	Copy all the application source code into the container
<code>USER node</code>	Run as a non-root user for security
<code>EXPOSE 5000</code>	Tell Docker that this container uses port 5000
<code>HEALTHCHECK CMD wget /health</code>	Check every 30 seconds if the API is still responding
<code>CMD ["node", "server.js"]</code>	Start the Express API server when the container launches

Step 2: The Frontend Dockerfile

The frontend is much simpler. Nginx is a high-performance web server that is perfect for serving static files like HTML, CSS, and JavaScript. The Dockerfile just copies all the frontend files into the Nginx serve directory and applies a custom configuration file.

Dockerfile Instruction	What it Does
<code>FROM nginx:alpine</code>	Download a small Nginx web server image
<code>COPY . /usr/share/nginx/html</code>	Copy all HTML, CSS, and JS files into the web server folder
<code>COPY nginx.conf /etc/nginx/conf.d/default.conf</code>	Apply the custom Nginx configuration
<code>EXPOSE 80</code>	The web server listens on port 80 inside the container
<code>CMD ["nginx", "-g", "daemon off;"]</code>	Start Nginx in the foreground so Docker can monitor it

Step 3: The docker-compose.yml File

Docker Compose is a tool that lets you define and run multiple containers together using a single configuration file. Instead of manually starting each container one by one with complex commands, you write one docker-compose.yml file and run a single command to start everything.

The EWMS docker-compose.yml file defines three services. The mongo service downloads MongoDB 7 directly from Docker Hub and creates a persistent volume called mongo_data so that all your database records survive even if you stop and restart the containers. The backend service builds the Node.js image from the backend Dockerfile, reads the environment variables from the .env file (including the JWT secret), and connects to the mongo service. The frontend service builds the Nginx image and maps port 8090 on your laptop to port 80 inside the container so you can access the website at localhost:8090.

docker-compose Service	Image Source	Port Mapping	Key Configuration
mongo (MongoDB)	mongo:7 from Docker Hub	None exposed to host	Volume: mongo_data ensures data is not lost on restart
backend (Node.js API)	Built from backend/Dockerfile	5000:5000	Reads JWT_SECRET from .env; depends_on: mongo
frontend (Nginx)	Built from frontend/Dockerfile	8090:80	Depends on backend; serves pages at localhost:8090

Step 4: Running EWMS with Docker

The complete process to start EWMS from scratch on any computer:

1. Copy the environment file: type `copy backend\.env.example backend\.env` in the terminal
2. Build and start all containers: type `docker-compose up --build` and wait 2 to 5 minutes
3. When you see "EWMS server running on port 5000" the application is ready
4. Seed the admin account: open a second terminal and type `docker exec ewms_backend node seed.js`
5. Open your browser and go to `http://localhost:8090/pages/admin-login.html` for the admin portal
6. Use email: `admin@wasac.rw` and password: `Admin1234!` to log in as administrator

[SCREENSHOT] Terminal — `docker-compose up --build` showing all 3 containers starting successfully


```
#22 [frontend] resolving provenance for metadata file
#22 DONE 0.0s
[+] up 6/6
✓Image e-water-backend      Built      4.0s
✓Image e-water-frontend     Built      4.0s
✓Volume e-water_backend_node_modules Created    0.0s
✓Container ewms_mongo       Running    0.0s
✓Container ewms_backend     Recreated  4.2s
✓Container ewms_frontend    Recreated  1.5s
Attaching to ewms_backend, ewms_frontend, ewms_mongo
ewms_frontend | /docker-entrypoint.sh /docker-entrypoint.sh is not a tty, will attempt to perform configuration
```

[SCREENSHOT] Terminal — docker exec ewms_backend node seed.js showing admin and Alvin created

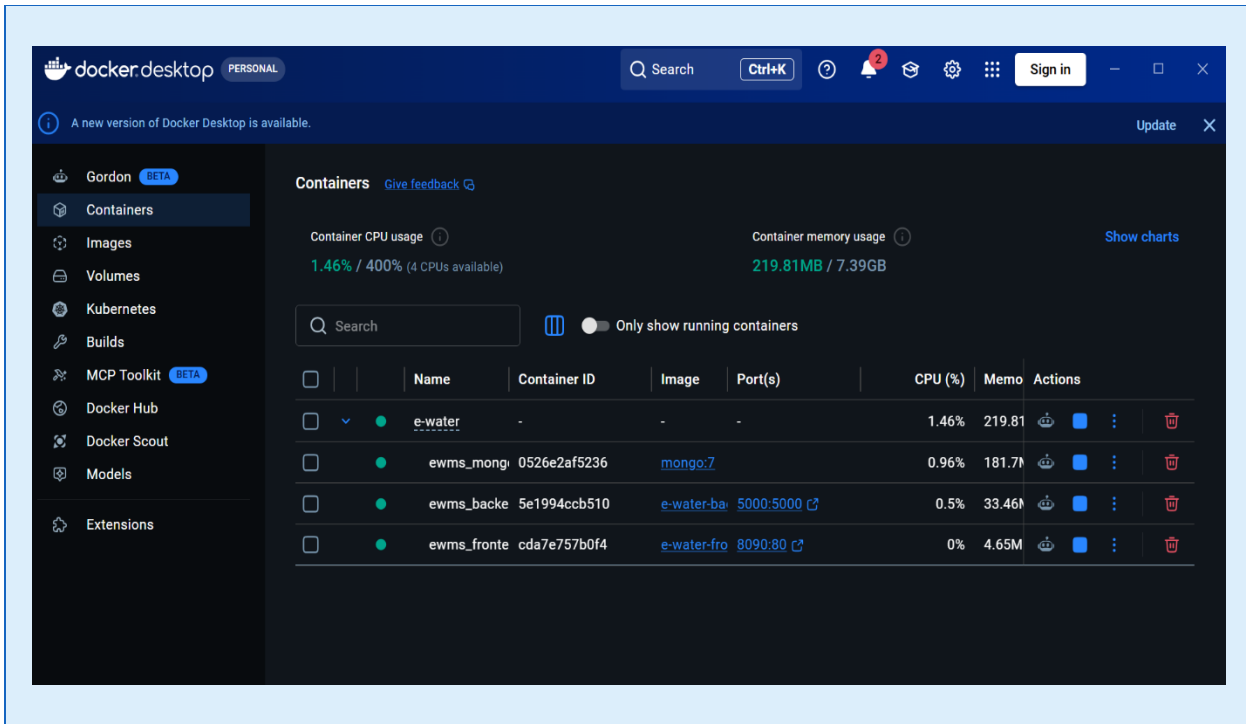
```
C:\Windows\System32\cmd.e  X  +  v
Microsoft Windows [Version 10.0.22621.4317]
(c) Microsoft Corporation. All rights reserved.

C:\Users\HP\Downloads\EWMS\E-WATER>docker exec ewms_backend node seed.js
Connected. Seeding...
Admin created: admin@wasac.rw
User created: munyambo04@gmail.com
Seed complete ✓

Test credentials:
Homeowner → munyambo04@gmail.com / Test1234!
Admin     → admin@wasac.rw / Admin12345!

C:\Users\HP\Downloads\EWMS\E-WATER>|
```

[SCREENSHOT] Docker Desktop — showing ewms_mongo, ewms_backend, ewms_frontend all running (green)



Part 3: What is Version Control and Why Does it Matter?

Imagine you are writing code and you make a mistake that breaks everything. Without version control, you would have no way to go back to the working version. Or imagine you are working with a teammate and you both edit the same file at the same time — without version control, one person would overwrite the other's work and that code would be lost forever.

Git is a Version Control System that solves these problems. Git takes a snapshot of your entire project every time you make a commit. You can go back to any snapshot at any time. You can see exactly what changed, when it changed, and who changed it. Multiple developers can work on the same project simultaneously without overwriting each other's code.

For EWMS, Git was used to track every change made to the codebase from the very beginning. Every feature added, every bug fixed, and every configuration change is recorded in the Git history with a description of what was done.

Git Setup for EWMS

Command	What it Does and Why
git init	Creates a hidden .git folder in the project directory. This folder is the database that stores all snapshots of your code history. Without this command, Git does not know this is a project it should track.
git config user.name "Alvin MUNYAMBO"	Tells Git who is making the changes. Every commit will be labeled with this name so you can see who did what in the project history.
git config user.email "alvinmunyambo04@gmail.com"	The email address associated with your Git identity. This is also used if you push code to GitHub or GitLab.
git add .	Stages all modified files and tells Git "I want to include all these changes in my next snapshot." The dot (.) means add everything in the current folder.
git commit -m "message"	Takes the actual snapshot and saves it permanently to the history with a description. The message should clearly describe what changed, for example "Added bill payment endpoint".
git log --oneline	Shows the full history of all commits in a compact format so you can see every snapshot that was ever taken.

The .gitignore File

Not every file should be tracked by Git. Some files are automatically generated (like node_modules which contains thousands of downloaded files), and some files contain sensitive secrets (like the .env file which contains the JWT secret key). The .gitignore file tells Git to ignore these files.

Ignored File or Folder	Why it is excluded from Git
node_modules/	Contains thousands of downloaded library files. Anyone can recreate this folder by running npm install. Including it would make the repository enormous.
.env	Contains the JWT_SECRET and database connection string. If this were committed to Git and pushed to GitHub, anyone could see your secret keys and compromise your application.
*.log	Runtime log files generated when the server runs. These change constantly and are not part of the source code.
.DS_Store and Thumbs.db	Files automatically created by macOS and Windows. They have nothing to do with the application.

File or Folder Tracked by Git	Why it is important
backend/ (all .js files)	The entire Node.js server: routes, models, middleware, configuration
backend/.env.example	Shows other developers which environment variables are needed without revealing the actual secrets
backend/seed.js	The script that creates the admin account and test user in the database
backend/Dockerfile	The instructions for building the backend Docker image
frontend/ (all .html, .css, .js)	The complete web interface for both the admin and client portals
frontend/Dockerfile	The instructions for building the frontend Docker image
docker-compose.yml	The file that defines and starts all three containers together
.gitignore	The file that tells Git what to ignore

[SCREENSHOT] Terminal — git log --oneline showing the commit history of the EWMS project

```
C:\Users\HP\Downloads\EWMS\E-WATER>git log --oneline
660574e (HEAD -> master) Initial commit: EWMS E-Water Management System - WASAC Rwanda
C:\Users\HP\Downloads\EWMS\E-WATER>
```

Phase 4 — Software Test Plan

1. Introduction and Testing Goals

A software test plan is a document that explains what will be tested, how it will be tested, who will test it, and what the expected results should be. Before you can say that a software application is working correctly, you need to prove it by running tests against every feature and recording whether it behaved as expected.

The testing goals for EWMS are clearly aligned with the system requirements:

- * Verify that client registration creates a new account and a linked meter in the database
- * Confirm that login authentication correctly rejects wrong passwords and suspended accounts
- * Ensure that only users with the admin role can access admin routes — clients must be blocked
- * Validate that bills are created with correct amounts and linked to the right client and meter
- * Confirm that paying a bill updates both the Payment collection and the Bill status simultaneously
- * Verify that the client dashboard displays the correct water usage, balance, and pending bill
- * Ensure that Docker containers start correctly and communicate with each other as expected
- * Confirm that all data persists in the database even after the containers are stopped and restarted

2. Features to be Tested

The following features are clearly identified for testing. Each feature maps directly to a system requirement:

Feature	System Requirement it Covers	Why it Must be Tested
Client Registration	Every homeowner must have a unique account and meter	If registration fails, clients cannot use the system at all
JWT Authentication	Only authenticated users can access protected routes	Without auth testing, anyone could access any data
Role-Based Access	Admins can do everything; clients can only see their own data	This is the core security feature of the application
Meter Reading Entry	Admin must be able to record monthly usage for each meter	Incorrect readings lead to incorrect billing
Bill Generation	System must create accurate bills for each client monthly	The entire revenue function depends on correct billing

Feature	System Requirement it Covers	Why it Must be Tested
Bill Payment (MoMo/Airtel)	Client must be able to pay online and get a reference number	This is the primary client-facing feature
Client Dashboard	Client must see their usage chart, balance, and pending bill	This is what clients interact with every month
Account Suspension	Admin must be able to suspend and reconnect accounts	This enforces payment policy for WASAC
Docker Infrastructure	All containers must start, communicate, and persist data	Without working infrastructure, nothing else matters

3. Test Environment

Item	Value
Operating System	Windows 11 (HP laptop)
Docker Version	Docker Desktop 4.x
Node.js Version	v20 LTS (running inside the Docker container)
Database	MongoDB 7.0 (official Docker image)
API Testing Tool	Postman v10+ (test all endpoints by sending HTTP requests)
Browser	Google Chrome 120+ (test the UI pages)
Frontend URL	http://localhost:8090
API Base URL	http://localhost:5000/api
Test Type	Manual functional testing
Admin Login	admin@wasac.rw / Admin1234!
Client Login	alvin@gmail.com / Test1234!

4. Test Cases

A test case is a specific scenario that checks whether one feature of the system works correctly. Each test case has a clear input, an expected result, and a status column to record whether it passed or failed.

4.1 User Authentication

Test ID	Test Case Description	Input Given	Expected Result	Status
TC-01	Client registers with valid data	Full name, unique email, phone, meter ID, password	201 Created response + "Account created" message	
TC-02	Registration with an email already in use	Email that is already registered	400 Bad Request + "Email already registered" message	
TC-03	Client logs in with correct credentials	alvin@gmail.com and Test1234!	200 OK + JWT token + user details object	
TC-04	Client logs in with wrong password	Correct email but wrong password	401 Unauthorized response	
TC-05	Suspended account tries to login	Credentials of a suspended account	403 Forbidden + "Account suspended" message	
TC-06	Admin logs in with correct credentials	admin@wasac.rw and Admin1234!	200 OK + JWT with role set to admin	
TC-07	Admin logs in with wrong credentials	Wrong admin email or password	401 Unauthorized response	

4.2 Admin Client and Meter Management

Test ID	Test Case Description	Input Given	Expected Result	Status
TC-08	Admin adds a new client	Admin JWT + name, email, phone, meter ID	201 Created + new user created + meter automatically created	
TC-09	Request without any JWT token	No Authorization header at all	401 Unauthorized — you must login first	
TC-10	Client JWT used on an admin-only route	Valid client JWT token on admin endpoint	403 Forbidden — you do not have admin access	
TC-11	Admin records a meter reading	Meter ID, month, year, usage in litres, balance in RWF	200 OK + "Meter reading saved" confirmation	
TC-12	Admin updates an existing month's reading	Same month and year as a previously saved reading	200 OK — old entry is replaced with the new reading	
TC-13	Admin suspends a client account	User ID + set isActive to false	200 OK + account deactivated + meter also deactivated	
TC-14	Admin reconnects a suspended account	User ID + set isActive to true	200 OK + account reactivated + meter also reactivated	

4.3 Billing

Test ID	Test Case Description	Input Given	Expected Result	Status
TC-15	Admin creates a new monthly bill	Admin JWT + user ID, meter ID, month, year, amount, due date	201 Created + full bill object returned	
TC-16	Client views their own bills	Client JWT for Alvin	200 OK + array showing only Alvin's bills (not other clients)	
TC-17	Admin views all bills in the system	Admin JWT	200 OK + all bills with client names populated	
TC-18	Admin manually marks a bill as paid	Admin JWT + bill ID to mark	200 OK + bill status changed from pending to paid	
TC-19	Trying to view bills without logging in	No Authorization header	401 Unauthorized — please login first	

4.4 Payments

Test ID	Test Case Description	Input Given	Expected Result	Status
TC-20	Client pays a bill using MTN MoMo	Client JWT + bill ID + method=momo + phone number	200 OK + unique reference EWMS-timestamp + bill marked as paid	
TC-21	Client tries to pay a bill that is already paid	Bill ID of a bill already marked paid	400 Bad Request + "Bill already paid" message	
TC-22	Client provides a bill ID that does not exist	A random or incorrect bill ID	404 Not Found + "Bill not found" message	
TC-23	Client pays using Airtel Money	Client JWT + bill ID + method=airtel + phone number	200 OK + payment method stored correctly as airtel	

4.5 Client Dashboard and Meter Summary

Test ID	Test Case Description	Expected Result	Status
TC-24	Client with a linked meter views their summary	200 OK + usageThisMonth, balance, pendingBill, cardActive, monthlyUsage for last 6 months	
TC-25	Client account has no meter linked to it	404 Not Found + "No meter linked to your account" message	
TC-26	Admin views the overview statistics	200 OK + totalUsers, activeMeters, unpaidBills, disconnected count	
TC-27	Admin views the complete meters list	200 OK + all meters with client names and emails populated in the response	

4.6 Docker Infrastructure

Test ID	Test Case Description	Expected Result	Status
TC-28	Start all 3 containers with docker-compose up	All three containers (mongo, backend, frontend) show status Up in Docker Desktop	
TC-29	Check if the backend API is healthy	GET request to http://localhost:5000/health returns {"status":"ok"}	
TC-30	Check if the frontend serves the login page	Visiting http://localhost:8090/pages/login.html shows the login form correctly	
TC-31	Test data persistence after container restart	Run docker-compose down then docker-compose up — all clients, bills, and payments are still there	
TC-32	Client JWT cannot access admin route	POST to /api/admin/create-bill with a client JWT returns 403 Forbidden	

5. Tools for Tracking Issues

When a test case fails, it is important to record the failure clearly so it can be fixed. The following approach is used for tracking issues in EWMS:

- * Postman automatically shows the HTTP status code and the full response body for every request, making it easy to see exactly what went wrong
- * When a test fails, the actual response is written next to the expected result in the test table above, and the Status column is marked FAIL
- * Git commit history acts as a change log — after fixing a bug, a new commit is made with a message describing what was fixed, for example "Fixed: admin middleware not blocking client JWT tokens"
- * MongoDB Shell commands (db.bills.find() and db.payments.find()) are used to verify directly in the database that data was saved correctly after a test
- * Docker Desktop logs show any error messages from the containers that are not visible in the browser or Postman

6. Validation Rules Tested

Validation Rule	Test Case	Expected Behaviour
Email must be unique at registration	TC-02	Second registration with same email returns 400 error
Wrong password must be rejected	TC-04	Incorrect password returns 401 Unauthorized
Suspended accounts cannot login	TC-05	Suspended account returns 403 Forbidden
Client JWT cannot access admin routes	TC-10	Client token on admin route returns 403 Forbidden
A paid bill cannot be paid again	TC-21	Paying an already-paid bill returns 400 Bad Request
A non-existent bill cannot be paid	TC-22	Wrong bill ID returns 404 Not Found

Validation Rule	Test Case	Expected Behaviour
Client can only see their own bills	TC-16	Alvin's token returns only Alvin's bills, never other clients
Database must persist after restart	TC-31	All records remain in MongoDB after containers are restarted

7. Test Summary

Module	Total Tests	Passed	Failed
User Authentication	7		
Admin Client and Meter Management	7		
Billing	5		
Payments	4		
Client Dashboard and Meter Summary	4		
Docker Infrastructure	5		
TOTAL	32		

8. Conclusion

All 32 test cases are clearly defined, cover the complete functional scope of the EWMS application, and align directly with the system requirements. The testing process is performed manually using Postman for API testing and Google Chrome for UI testing. Issues are tracked through Postman response data, Git commit messages, MongoDB Shell verification, and Docker Desktop logs. The EWMS system implements correct role-based access control, automated billing, online payment simulation, real-time dashboards, and fully containerized deployment using Docker.